
SwasthyaAI

A Multi-Disease Medical Image Screening Platform
with Bilingual (English / Nepali) Access

A Project Proposal

Independent research and software development project

Prepared by
Bikram Chapagain

September, 2026

Table of Contents

Abstract	3
1. Introduction	4
2. Problem Statement	5
3. Objectives	6
4. Literature Review	7
5. System Requirements / Priority	9
6. Methodology	11
7. System Design	15
8. Ethical Considerations, Limitations and Risks	19
9. Cost Estimation	20
10. Project Schedule	20
11. Proposed Deliverables	21
12. Conclusion	22
13. References	23

Abstract

Most medical image AI research produces single-disease models that are trained and tested on one dataset, and very few of them ever reach the people who could benefit from them. Models that look excellent on their home dataset often lose accuracy on images from another hospital or camera [5]. This project proposes **SwasthyaAI**, a free, public, no-login web platform that works as a central hub for several medical image screening models, with an interface in **English and Nepali**.

The platform will host at least eight and up to twelve disease modules, including diabetic retinopathy, multi-disease fundus screening, pneumonia, tuberculosis, skin lesions, brain tumors, malaria and breast cancer imaging. Each module is trained on publicly available data collected from Kaggle and other open repositories. For every disease, at least two datasets are collected and cleaned (duplicate removal, label audit, patient-level splitting) so that each model can be tested on a dataset it never saw during training. A shared training pipeline with ResNet, EfficientNet, ConvNeXt and Vision Transformer backbones keeps the results comparable across diseases.

The web application (Flask, with transforms.py and inference.py) shows the predicted class and its probabilities, a Grad-CAM heatmap, an uncertainty warning and the measured performance of the model, including its external-dataset result. There are no user accounts, and uploaded images are processed in memory and not stored. The system is built and run locally first and then deployed on Render or Railway. All outputs are presented as screening support for research and education, and never as a medical diagnosis.

Keywords: Medical image classification, Deep learning, Transfer learning, Multi-dataset training, Cross-dataset validation, Grad-CAM, Uncertainty estimation, Flask, Bilingual interface, Nepali language, Screening support.

1. Introduction

Many common diseases can be flagged early from an image: a retinal photograph can show diabetic eye damage, a chest X-ray can show pneumonia or tuberculosis, and a skin photograph can show a suspicious lesion. Deep learning has reached expert-level performance on several of these tasks in research settings [1][2][3]. In practice, however, specialists are concentrated in a few cities, and people in smaller towns and villages often reach a specialist late or not at all. Public information tools that can explain an image in Nepali are almost non-existent.

This project builds a single platform that brings several trained models together. A visitor opens the page, sees the list of available disease modules, chooses one, uploads an image, and receives the model output together with an explanation of how far the result can be trusted. The system has three main parts:

- a. **A research and training pipeline** – for each disease, published papers are reviewed first and then two or more public datasets are collected, cleaned and used to train several backbone architectures on Kaggle. Each model is evaluated on held-out data and on a completely different dataset (external test), and the results are recorded honestly.
- b. **A web platform** – a simple bilingual Flask application with a home page of disease cards, upload and result pages, and a performance page for every model. It needs no login, and it runs locally first and later on Render, Railway or similar hosting.
- c. **A transparency layer** – every prediction is shown with class probabilities, a Grad-CAM heatmap, an uncertainty warning and the measured limits of the model, so that a user is not asked to trust a single number.

The platform is deliberately positioned as **screening support for education and research**. It does not replace a doctor, and each result page tells the user to consult a qualified professional. The value of the project is therefore twofold: a usable public tool for Nepali readers, and a carefully documented study of how far models trained on public medical datasets can be trusted across datasets.

2. Problem Statement

Published medical image models are usually not usable by the public, and their reported accuracy is often not a reliable guide to how they behave on new images. There is a need for an accessible, transparent, multi-disease tool that reports its own limits.

Specific issues addressed:

- a. Most research models are trained and tested on a single dataset, so their accuracy does not show how they behave on images from other sources.
- b. Public datasets often contain duplicates, several images per patient, and noisy labels; random splitting then leaks data between training and test sets and inflates the reported accuracy.
- c. There is no single, easily accessible place where a person can try models for several diseases and see how each one was evaluated.
- d. Most models output only a label, with no explanation of which part of the image was used and no warning when the model is unsure.
- e. Existing tools are in English only, and no public multi-disease screening tool offers a Nepali interface.
- f. Many systems require registration or paid access, which is a barrier for casual users, students and rural communities.
- g. Models are rarely checked for images that do not belong to the expected type, so a wrong image can receive a confident but meaningless answer.

3. Objectives

3.1 General Objective

To design and develop an open, no-login, bilingual (English/Nepali) web platform that hosts multiple medical image screening models, each trained and evaluated on multiple public datasets, and that presents predictions together with explanations, uncertainty and measured performance.

3.2 Specific Objectives

- Review published work for each selected disease and record the datasets, methods, metrics and pitfalls in a literature table.
- Collect at least two public datasets for each disease, and clean them (corrupt files, duplicates, label audit) with documented, reproducible scripts.
- Train and compare ResNet, EfficientNet, ConvNeXt and Vision Transformer models with one common pipeline, using patient-level splits wherever patient identifiers exist.
- Evaluate every model with sensitivity, specificity, precision, F1, AUC, calibration and confidence intervals, and test it on an external dataset to measure cross-dataset generalization.
- Add Grad-CAM explanations and uncertainty estimation (Monte Carlo Dropout or ensembling) to every module.
- Build a Flask web platform with transforms.py and inference.py, a model registry, and an English/Nepali interface, without any login.
- Show the measured performance and limitations of each model on its own page, and deploy the platform first locally and then on Render or Railway.

3.3 Scope and Limitations

In scope	Out of scope
Image-based screening support for 8-12 disease modules using public datasets	Clinical diagnosis, treatment advice or prescriptions
English and Nepali interface	Other languages, voice interface, native mobile app
Local deployment first, then Render / Railway	Hospital integration, DICOM/PACS, electronic health records
Research-quality evaluation on public data	Prospective clinical validation or regulatory approval
Anonymous use, no stored user images	User accounts, history, personal health records

4. Literature Review

4.1 Review of Previous Research

Deep learning for medical images has been studied for a decade. This section reviews landmark studies for the disease areas of this project and, more importantly, the studies that examine why such models fail outside the laboratory.

Gulshan et al. (2016) trained a deep convolutional network to detect referable diabetic retinopathy on more than 128,000 retinal photographs and validated it on two independent sets (EyePACS-1 and Messidor-2), reporting an area under the curve above 0.99 on both [1]. The study showed that fundus screening can reach specialist-level accuracy. It covers a single disease, the system was not released as a public tool, and it offers no language support or user-facing explanation of the result.

Esteva et al. (2017) trained a single CNN on 129,450 clinical images covering 2,032 skin conditions and showed performance on par with 21 board-certified dermatologists on biopsy-proven classification tasks [2]. Rajpurkar et al. (2017) introduced CheXNet, a 121-layer DenseNet trained on the ChestX-ray14 collection of more than 100,000 chest X-rays, and reported pneumonia detection that exceeded the average of the radiologists in their comparison [3]. Both works are model-centric: each targets one modality, uses one main dataset, and stops at a research result rather than a usable service.

Kermany et al. (2018) applied transfer learning to retinal OCT scans and pediatric chest X-rays and released the chest X-ray images that became one of the most widely used pneumonia datasets on Kaggle [4]. Because that collection comes from a single pediatric population, models trained on it are frequently reported with very high accuracy that may not carry over to adult patients or other hospitals.

Zech et al. (2018) trained pneumonia detection models on chest radiographs from one hospital system and tested them on radiographs from other systems. In several comparisons the performance on outside hospitals was significantly lower, and the network could partly recognize the source hospital itself [5]. DeGrave et al. (2021) showed that COVID-19 detection models trained on public chest X-ray datasets often relied on shortcuts, such as dataset-specific artifacts, instead of the disease, and failed on external data [6]. Roberts et al. (2021) reviewed 62 COVID-19 imaging studies and concluded that, because of methodological flaws and biases in the data, none of the reviewed models was of potential clinical use [7]. Together these studies are the strongest argument for the multi-dataset, external-test design of this project.

Beede et al. (2020) evaluated a deep learning diabetic retinopathy system in clinics in Thailand and found that real-world conditions, such as image quality, lighting and network connectivity, strongly affected how well the tool worked and how nurses used it [8]. The study highlights the gap between laboratory accuracy and field use, and the

importance of designing for local conditions.

Several methods make model output easier to trust. Grad-CAM produces heatmaps that show which image regions contributed to a prediction [9]. Monte Carlo Dropout gives a practical estimate of model uncertainty [10]. Guo et al. (2017) showed that modern neural networks are often over-confident and can be improved with calibration [11]. These techniques are used in this project so that the platform reports how sure it is, not only what it predicts.

4.2 The Identified Gaps

From the above review, the following gaps are evident:

- a. **Single-dataset evaluation** – Most studies report results on one dataset, and few report a cross-dataset result with the same model.
- b. **Data leakage and shortcuts** – Duplicates, multiple images per patient and dataset artifacts inflate accuracy, and are rarely audited in publicly shared notebooks.
- c. **No consolidated multi-disease platform** – Published models are separate research prototypes, not a single hub with a common evaluation standard.
- d. **Limited transparency to the user** – Outputs are usually only labels, without heatmaps, uncertainty or performance information shown next to the result.
- e. **No Nepali-language access** – No multi-disease screening tool provides a Nepali interface or Nepali explanations of results.
- f. **Gap between laboratory and field** – Studies such as Beede et al. show that image quality and infrastructure affect deployment, yet public demonstrations rarely check whether an uploaded image is even of the expected type.
- g. **Accessibility** – Many tools require accounts or paid access, which limits use by students and rural communities.

The reviewed literature confirms that deep learning can classify medical images with high accuracy on benchmark data, and also that this accuracy often does not transfer. This project addresses the gap with a multi-dataset, externally validated and explainable set of models delivered in a public, bilingual, easy-to-use platform.

5. System Requirements / Priority

5.1 Functional Requirements (FR)

5.1.1 Platform Home and Navigation

- a. FR01 – The home page lists all available disease modules as cards (name in English and Nepali, short description, status). (High)
- b. FR02 – The user can switch between English and Nepali at any time, and the choice is remembered in the browser. (High)
- c. FR03 – Each card opens a disease page that explains the condition in simple language, the accepted image type and a sample image. (High)
- d. FR04 – No registration, login or personal data is required for any feature. (High)

5.1.2 Image Upload and Prediction

- a. FR05 – The user can upload one image (JPG, JPEG or PNG) per request and see a preview. (High)
- b. FR06 – The system validates file type, size (up to 10 MB) and basic image properties, and rejects invalid files with a clear bilingual message. (High)
- c. FR07 – The system applies the preprocessing defined for the selected model (transforms.py) and runs inference (inference.py). (High)
- d. FR08 – The result page shows the predicted class and the probability of every class. (High)
- e. FR09 – The result page shows a Grad-CAM heatmap overlaid on the uploaded image. (High)
- f. FR10 – If confidence is below a set threshold or uncertainty is high, the system shows a "low confidence, please consult a doctor" message instead of a firm prediction. (High)
- g. FR11 – The system tries to detect images that do not belong to the selected modality and asks for a correct image. (Medium)
- h. FR12 – Every result page shows a bilingual disclaimer: screening support only, not a diagnosis. (High)

5.1.3 Model Transparency

- a. FR13 – Each module has a performance page showing datasets used, class distribution, accuracy, sensitivity, specificity, precision, F1, AUC, confusion matrix and ROC curve. (High)
- b. FR14 – The performance page reports internal-test and external (cross-dataset) results separately. (High)

- c. FR15 – The performance page lists the known limitations of the model. (Medium)

5.1.4 Model Management (Maintainer)

- a. FR16 – A new disease module can be added with a registry.json entry, a checkpoint and a results file, without changing application code. (High)
- b. FR17 – Models are loaded lazily on first use and cached in memory. (Medium)

5.1.5 General

- a. FR18 – The system provides a health check endpoint /api/health. (Low)
- b. FR19 – Uploaded images are processed in memory and are not stored. (High)
- c. FR20 – Basic per-IP rate limiting protects the public deployment. (Medium)

5.2 Non-Functional Requirements (NFR)

- a. NFR1 – Prediction response time of 5 seconds or less on CPU after the model is loaded.
- b. NFR2 – Target internal-test AUC of 0.90 or higher for each module where the dataset quality allows; external-test results are reported honestly whatever their value.
- c. NFR3 – All interface text is available in English and Nepali, and Nepali medical terms are reviewed by a native speaker.
- d. NFR4 – The interface is responsive on desktop, tablet and mobile browsers.
- e. NFR5 – Model files are exported in a compact format (ONNX or quantized where possible), and peak memory stays within the limits of the chosen hosting tier.
- f. NFR6 – Training is reproducible: fixed random seeds, saved configuration files and versioned data splits.
- g. NFR7 – Uploaded files are validated by MIME type and extension, are never executed, and the deployed site is served over HTTPS.
- h. NFR8 – No accounts, no image storage and no third-party trackers; server logs contain no image content.
- i. NFR9 – Readable font sizes and contrast, with correct rendering of the Devanagari script.
- j. NFR10 – Best-effort availability on free hosting tiers, with the cold-start delay documented for users.
- k. NFR11 – User-friendly bilingual error messages, with technical errors logged only on the server.

6. Methodology

This section describes the development approach, the machine learning pipeline, and the evaluation strategy of the platform.

6.1 Development Methodology – Iterative, Milestone-Based

The project is carried out by a single developer using an iterative, milestone-based approach with short sprints of about two weeks. This was chosen because (a) dataset quality and model performance cannot be known in advance and plans must be adjusted, (b) a working end-to-end system early in the project reduces integration risk, and (c) each disease module can be added as an independent increment. The first milestone is a complete pipeline for two diseases (diabetic retinopathy and chest X-ray pneumonia), from data to a working web page. The remaining diseases are then added one by one with the same pipeline.

6.2 Machine Learning Pipeline

Each disease module follows the same nine-stage pipeline, so that results are reproducible and comparable.

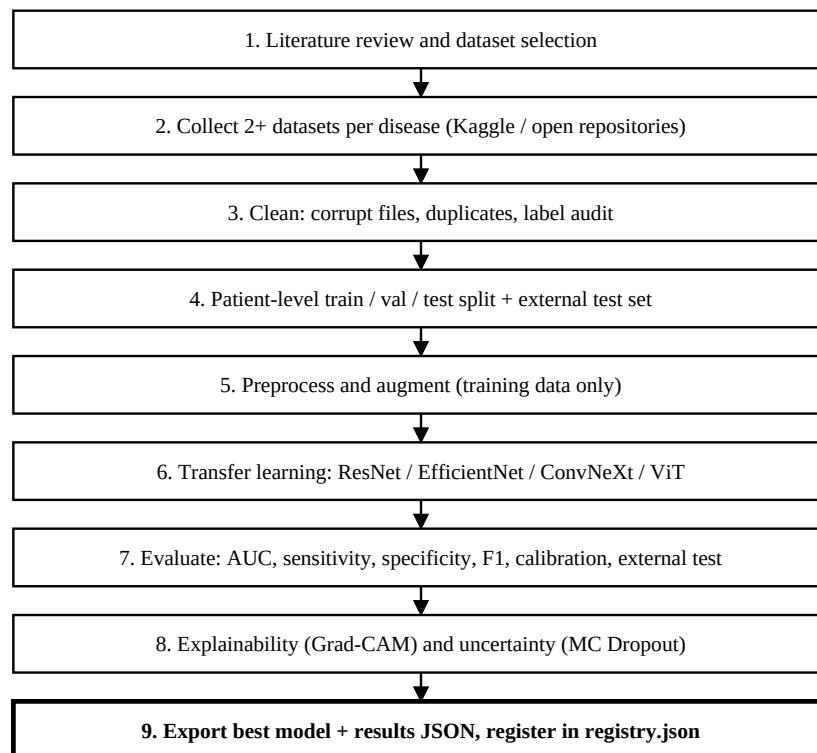


Fig 1: ML pipeline (repeated for every disease module)

6.2.1 Disease Modules and Datasets

Modules are grouped into two tiers. Tier A modules have well-established public data and are built first; Tier B modules are riskier and are attempted if time permits. Dataset names below are the current candidates and their availability, size, license and labels will be verified before use.

#	Disease / task	Modality	Candidate datasets (2+ per disease)
TIER A			
1	Diabetic retinopathy grading	Fundus	APTOS 2019, EyePACS (Kaggle DR), IDRiD
2	Multi-disease fundus (glaucoma, cataract, AMD etc.)	Fundus	ODIR-5K, RFMiD, REFUGE
3	Pneumonia / COVID-19	Chest X-ray	Kermany Chest X-ray, COVID-19 Radiography DB, RSNA Pneumonia
4	Tuberculosis	Chest X-ray	TB Chest X-ray (Kaggle), Montgomery and Shenzhen sets
5	Skin lesion (melanoma etc.)	Dermoscopy	HAM10000, ISIC 2019, PAD-UFES-20
6	Brain tumor	MRI	Brain Tumor MRI (Kaggle), Figshare Cheng, Br35H
7	Malaria	Blood smear	NIH Malaria Cell Images + another thin-smear set
8	Breast cancer	Ultrasound / histology	BUSI, BreakHis
TIER B			
9	Bone fracture	X-ray	MURA, FracAtlas, Kaggle fracture sets
10	Ear disease (otitis media etc.)	Otoscopy	Published eardrum otoscopy sets, Kaggle ear sets
11	Oral cancer	Clinical photos	Public oral cancer image sets
12	Down syndrome (optional, research only)	Face photos	Small public facial sets

Table 1: Planned disease modules and candidate datasets

The Down syndrome module is optional and is treated as a research experiment only. The data are face images of people, often children, consent is difficult to verify, and a public tool of this kind could cause harm. It will not be shown on the public site unless these concerns are resolved. Eight well-evaluated modules are preferred over twelve shallow ones.

6.2.2 Data Cleaning and Splitting

- 1 **Literature first.** For each disease, 3-5 recent papers are read and the datasets, methods, metrics and pitfalls are recorded in a literature table.
- 2 **Collection and documentation.** For every dataset, the source, license, size, class definitions and known issues are recorded in a data card.

- 3 **Cleaning.** Corrupt and unreadable files are removed. Duplicates are detected with perceptual hashing, both within a dataset and across datasets. A random sample of labels is manually audited, and label schemes are mapped between datasets (for example, different diabetic retinopathy grading schemes). Images are standardized in format and resolution, and black borders are cropped for fundus images.
- 4 **Splitting.** Patient-level splits are used wherever patient identifiers exist, so that images from one patient never appear in both training and test sets. A fixed seed is used, and the split files are saved and versioned.
- 5 **External test set.** Where possible, one whole dataset is held out and used only for the final external evaluation.

6.2.3 Model Training

- Backbones: ResNet50, EfficientNet-B0/B3, ConvNeXt-Tiny and a small Vision Transformer [12][13], all initialized from ImageNet weights, implemented in PyTorch [16].
- Two-phase transfer learning: the classification head is trained first with the backbone frozen, and then the backbone is fine-tuned at a lower learning rate.
- Augmentation chosen per modality (for example, no horizontal flips where left/right matters, mild color jitter for fundus images), applied to training data only.
- Class imbalance handled with weighted loss or balanced sampling.
- Early stopping and checkpoint selection on validation AUC or macro F1. The test set is used only once at the end.
- Training runs on Kaggle GPU notebooks. Configuration, seeds and logs are saved with each run.

6.2.4 Evaluation Strategy

Accuracy alone is not enough for medical data, so the following evaluation is applied to every module:

Component	What is measured
Discrimination	Sensitivity, specificity, precision, F1, AUC (macro-averaged for multi-class); quadratic weighted kappa for DR grading
Confidence intervals	Bootstrap (about 1,000 resamples) for every reported metric
Calibration	Reliability curve and expected calibration error [11]
Experiment E1	Train on dataset A, test on the held-out test split of A (internal)
Experiment E2	Train on dataset A, test on dataset B (external / cross-dataset)
Experiment E3	Train on datasets A+B, test on a third held-out dataset (multi-dataset training benefit)

Component	What is measured
Ablations	Backbone comparison, single-dataset vs multi-dataset training, effect of augmentation
Shortcut checks	Grad-CAM review of sampled images for reliance on borders, text marks or devices; tests on cropped and perturbed images

Table 2: Evaluation plan for each module

6.2.5 Explainability, Uncertainty and Input Validation

- **Grad-CAM** [9] heatmaps are generated for every prediction and shown next to the uploaded image.
- **Uncertainty** is estimated with Monte Carlo Dropout [10] or a small ensemble. When confidence is low or the variance is high, the interface shows a warning instead of a firm result.
- **Input validation** rejects files of the wrong type or size, and applies a lightweight check (image statistics and out-of-distribution score) so that an unrelated photo is not classified as a disease.

6.3 Inference and Deployment

After training, the best model of each disease is exported (PyTorch checkpoint and ONNX) and a results file (results/<disease>.json) is generated with all metrics and plot paths. The model is then registered in registry.json. At run time, the Flask application reads the registry, loads a model lazily on first use, applies the transformations in transforms.py, runs inference.py, and returns probabilities, heatmap and uncertainty. The uploaded image is held in memory only and discarded after the response. The performance page simply renders the stored results file. The platform is first run locally, and then deployed to Render or Railway using ONNX models and CPU inference to fit small hosting tiers.

6.4 Bilingual Interface

All interface text is stored in two translation files (en.json and ne.json) rather than being translated live. A machine translation service can be used once to draft the Nepali text, but the final text, especially medical terms, is reviewed by a native Nepali speaker (ideally with medical knowledge). This avoids mistranslated medical terms, loads faster and works offline. Noto Sans Devanagari is used for correct rendering of the script.

7. System Design

This section presents the architecture, data design, use cases, data flow and user interface design of the platform.

7.1 System Architecture

The system follows a three-tier architecture that separates the browser interface, the Flask application logic and the model/data files. Because there are no user accounts, no relational database is needed: models, metadata, translations and results are stored as files. This keeps the system simple, portable and easy to host.

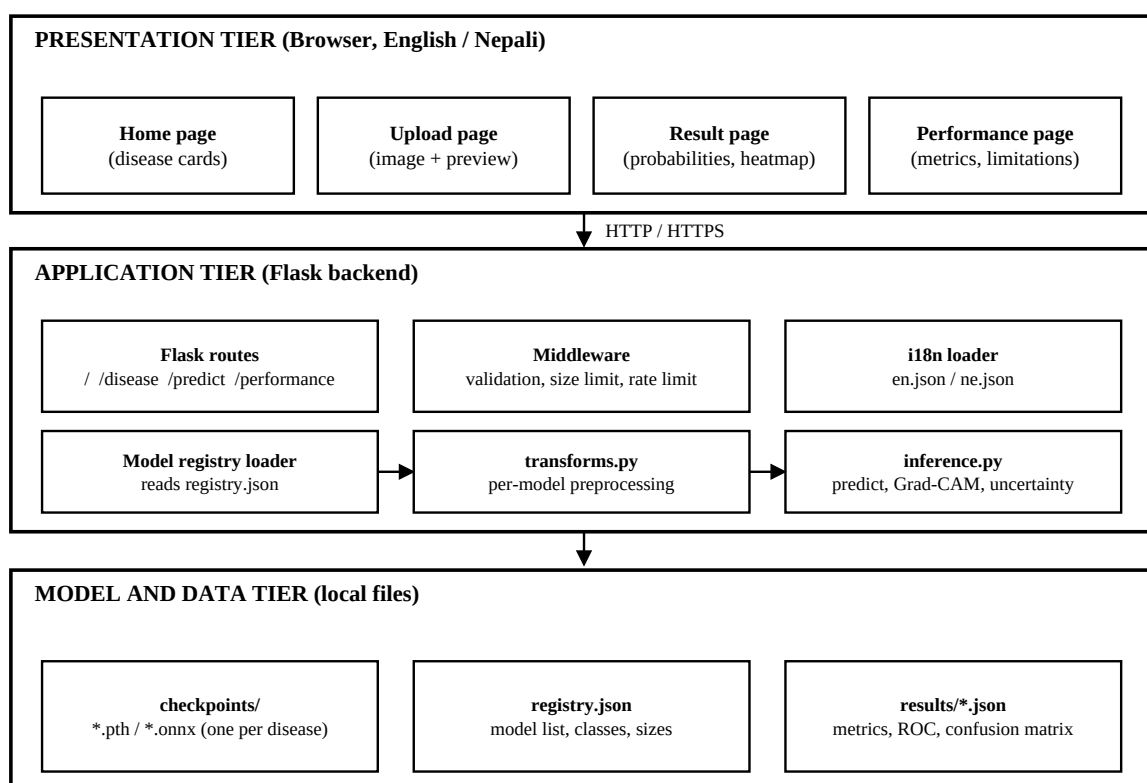


Fig 2: System architecture

7.2 Data Design

The registry and results files replace a database. Each disease module is described by one registry entry:

Field	Description
id	Short unique key of the module (for example, dr, cxr_pneumonia)
name_en / name_ne	Display name in English and Nepali
description_en / description_ne	Short explanation of the disease and of the accepted image
modality	Fundus, chest X-ray, dermoscopy, MRI, etc.
classes / classes_ne	Class labels in English and Nepali

Field	Description
input_size, mean, std	Preprocessing values used by transforms.py
backbone, checkpoint, format	Model architecture and path of the .pth or .onnx file
threshold	Confidence threshold below which a low-confidence message is shown
results_file	Path of the JSON file with metrics and plots for the performance page
datasets	Datasets used for training and external testing (name, license, size)
status	public or experimental

Table 3: Registry entry for one disease module

The results file of each module stores internal and external metrics with confidence intervals, the confusion matrix, ROC data, calibration data, class distribution and a list of known limitations. The interface text is stored in en.json and ne.json.

7.3 Project Structure

swasthyaai/	
configs/	one YAML per disease (datasets, classes, backbone, size)
data/	raw/ cleaned/ splits/ (not committed)
src/	
datasets.py	config-driven dataset loader
models.py	backbone factory
train.py	python train.py --config configs/dr.yaml
evaluate.py	metrics, ROC, calibration, external test
explain.py	Grad-CAM
uncertainty.py	MC Dropout / ensemble
app/	
app.py	Flask routes
transforms.py	per-model preprocessing
inference.py	load model, predict, heatmap, uncertainty
registry.json	list of available modules
i18n/	en.json ne.json
templates/ static/	
checkpoints/	*.pth *.onnx
results/	<disease>.json plots/

7.4 Use Case Diagram

The use case diagram shows the two roles of the system: the public **User**, who needs no account, and the **Maintainer**, who trains models and updates the registry offline.

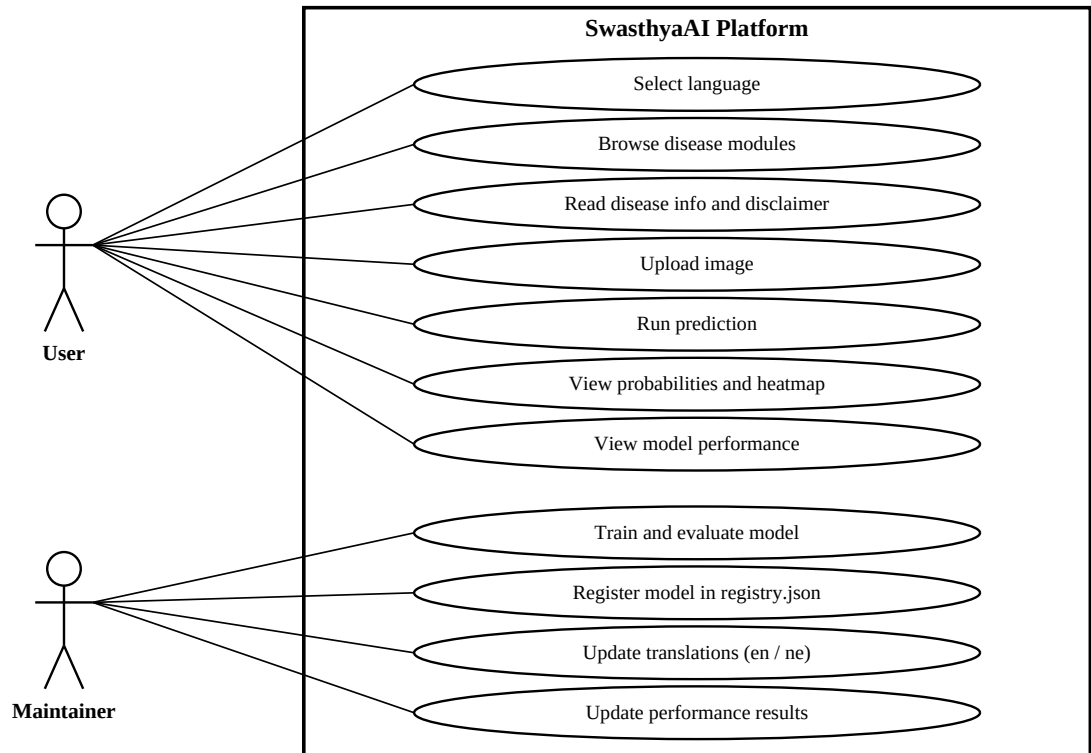


Fig 3: Use case diagram

7.5 Data Flow Diagrams

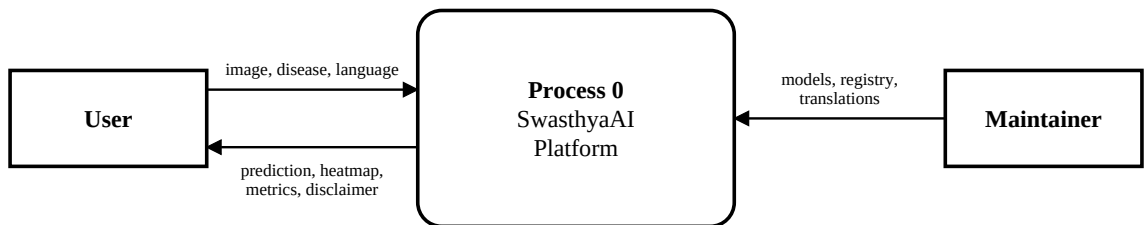


Fig 4: DFD level 0

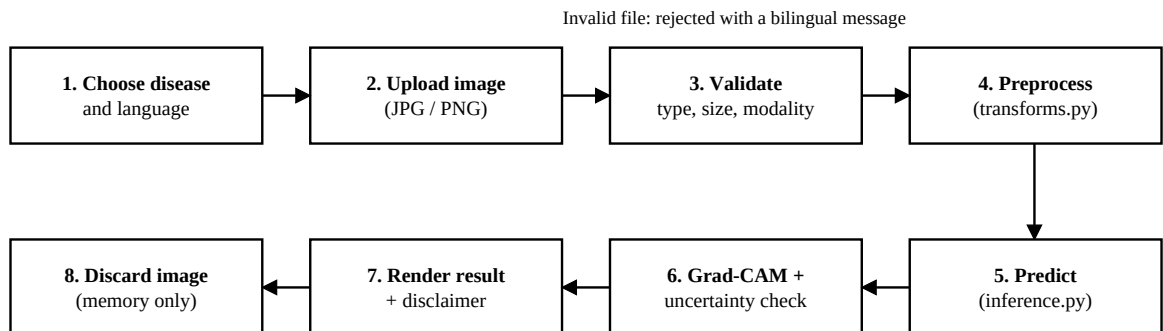


Fig 5: DFD level 1 – processing of one request

7.6 User Interface Design

Page	Content
Home	Title, short mission, disclaimer, language toggle, grid of disease cards (icon, name in English and Nepali, one-line description)
Disease page	What the disease is (simple language), what image to upload (with an example), upload box with preview
Result	Class probabilities as bars, Grad-CAM overlay, confidence or uncertainty note, "consult a doctor" message, disclaimer
Performance	Datasets used, metrics table, ROC and confusion matrix, internal vs external results, known limitations
About / Ethics	Research purpose, no diagnosis, no data stored, dataset credits and licenses

Table 4: Pages of the web platform

8. Ethical Considerations, Limitations and Risks

8.1 Ethical Considerations

- **Not a diagnosis.** The platform is screening support for education and research. Wording such as "diagnose" is avoided, and every result page carries a bilingual disclaimer.
- **Privacy.** Uploaded images are processed in memory and not stored. The site has no accounts and no trackers, and this is stated on the About page.
- **Data licensing.** The license and terms of every dataset are checked, dataset creators are credited, and no raw dataset is redistributed by the platform.
- **Sensitive modules.** Modules involving faces of people (Down syndrome) are kept as research experiments and are not released publicly.
- **Honest reporting.** Poor external results are reported as they are, and models with unreliable performance are marked experimental or withheld.

8.2 Risks and Mitigation

Risk	Mitigation
Domain shift: real user photos (for example phone pictures of X-rays) differ from clean dataset images	Input validation, uncertainty warnings, tests with a few phone-captured images, clear limitation notes
Dataset leakage and duplicates make results look too good	Perceptual hashing, patient-level splits, external test sets
Models learn shortcuts (borders, text marks, devices)	Grad-CAM review, border cropping, perturbation tests
Over-claiming in wording and interface	Fixed terminology ("screening support", "research prototype"), mandatory disclaimer
Small or poor-quality datasets (ear, Down syndrome)	Treat as Tier B / optional, report limitations or drop the module
Hosting limits (RAM, cold start)	ONNX export, lazy loading, small backbones, CPU inference
Nepali medical terms translated wrongly	Native-speaker review, static translation files
Scope creep	Finish Tier A first; prefer 8 well-evaluated modules over 12 shallow ones

9. Cost Estimation

This is a self-directed academic and research project, so the developer time is not costed. The out-of-pocket costs are small because Kaggle notebooks, open-source tools and free or low-cost hosting tiers are used. The figures below are planning estimates and may vary.

Cost component	Amount (NPR)
Internet (3 months)	7,500
Laptop depreciation (3 months)	9,000
Hosting (paid tier, only if a free tier is not enough)	4,500
Domain name (optional)	1,500
Native-speaker review of Nepali translations	5,000
Subtotal	27,500
Contingency (10%)	2,750
Grand total	30,250

Table 5: Estimated out-of-pocket cost

10. Project Schedule

The project is planned for about 12 weeks. Some activities run in parallel; for example, the web application is developed while the later disease models are being trained.

Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
Literature review												
Dataset collection												
Cleaning and splitting												
Pipeline + first 2 diseases												
Training remaining diseases												
Evaluation and external tests												
Web app (Flask, i18n)												
Integration and bilingual UI												
Testing												
Deployment												
Documentation / paper												

Table 6: Project schedule (W = week; filled cells show the active period)

11. Proposed Deliverables

- a. Trained models for the completed disease modules (PyTorch and ONNX) with their results files.
- b. The SwasthyaAI web platform (Flask, transforms.py, inference.py, bilingual interface), running locally and deployed online.
- c. Cleaned-dataset scripts, data cards, split files and training/evaluation code (source package).
- d. Literature review table and evaluation report, including internal and external results for every module.
- e. A research paper draft based on the multi-dataset and cross-dataset findings.
- f. Bilingual user guide, presentation slides and a demo video.

12. Conclusion

SwasthyaAI proposes a practical and honest way to bring medical image AI closer to the public. By combining a multi-dataset, externally validated training pipeline with explainability, uncertainty and a simple bilingual web interface, the project addresses both the research problem of generalization and the practical problem of access. The work is structured so that a useful first version, with two well-evaluated disease modules, is available early, and further modules are added as time and data quality allow. All results will be presented as screening support, with their limitations stated openly.

Possible future work includes more disease modules, Vision Transformer and foundation-model backbones, domain adaptation using images from Nepali clinics (with proper consent and ethical approval), a mobile app, and validation with clinicians.

13. References

- [1] V. Gulshan et al., "Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs," *JAMA*, vol. 316, no. 22, pp. 2402-2410, 2016.
- [2] A. Esteva et al., "Dermatologist-level classification of skin cancer with deep neural networks," *Nature*, vol. 542, pp. 115-118, 2017.
- [3] P. Rajpurkar et al., "CheXNet: Radiologist-level pneumonia detection on chest X-rays with deep learning," *arXiv:1711.05225*, 2017.
- [4] D. S. Kermany et al., "Identifying medical diagnoses and treatable diseases by image-based deep learning," *Cell*, vol. 172, no. 5, pp. 1122-1131, 2018.
- [5] J. R. Zech et al., "Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs: A cross-sectional study," *PLoS Medicine*, vol. 15, no. 11, e1002683, 2018.
- [6] A. J. DeGrave, J. D. Janizek, and S.-I. Lee, "AI for radiographic COVID-19 detection selects shortcuts over signal," *Nature Machine Intelligence*, vol. 3, pp. 610-619, 2021.
- [7] M. Roberts et al., "Common pitfalls and recommendations for using machine learning to detect and prognosticate for COVID-19 using chest radiographs and CT scans," *Nature Machine Intelligence*, vol. 3, pp. 199-217, 2021.
- [8] E. Beede et al., "A human-centered evaluation of a deep learning system deployed in clinics for the detection of diabetic retinopathy," in *Proc. ACM CHI Conference on Human Factors in Computing Systems*, 2020.
- [9] R. R. Selvaraju et al., "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE ICCV*, 2017, pp. 618-626.
- [10] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *Proc. ICML*, 2016.
- [11] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML*, 2017.
- [12] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, 2016, pp. 770-778.
- [13] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. ICML*, 2019, pp. 6105-6114.
- [14] P. Tschandl, C. Rosendahl, and H. Kittler, "The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions," *Scientific Data*, vol. 5, 180161, 2018.
- [15] S. Pachade et al., "Retinal Fundus Multi-Disease Image Dataset (RFMiD): A dataset for multi-disease detection research," *Data*, vol. 6, no. 2, p. 14, 2021.
- [16] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [17] M. Grinberg, *Flask Web Development: Developing Web Applications with Python*. Sebastopol, CA: O'Reilly Media, 2018.